📂 Estructura del Repositorio Open Source: POM Agents Framework

Este repositorio contiene exclusivamente la "lógica cerebral" del Product Operating Model (POM). No incluye interfaces gráficas, sino los *System Prompts*, esquemas de herramientas (Tools/Skills) y flujos de trabajo para los 33 agentes de IA especializados.

Está diseñado para ser clonado e integrado fácilmente en cualquier framework de orquestación multi-agente (como LangGraph, CrewAI o Autoresearch).

🌳 Árbol de Directorios

```
pom-agents-framework/
├── .github/                    # (Opcional) Workflows de GitHub Actions
├── agents/                     # 🧠 Definiciones de los 33 Agentes por vertical
│   ├── 01_leadership/
│   │   ├── ceo_agent/
│   │   │   ├── system_prompt.md  # Rol, contexto y guardarraíles
│   │   │   └── tools.xml         # Esquemas planos de herramientas (Pattern 1)
│   ├── 02_product/
│   │   ├── cpo_agent/
│   │   ├── discovery_agent/
│   │   ├── product_ops_agent/
│   │   └── product_analytics/
│   ├── 03_design/
│   ├── 04_engineering/
│   ├── 05_data/
│   ├── 06_growth_gtm/
│   └── 07_business_ops/
│
├── workflows/                  # 🔄 Lógica de orquestación (Ejemplos ejecutables
│   ├── discovery_loop.py       # Ejemplo: CPO <-> Discovery <-> User Research
│   ├── delivery_pipeline.py    # Ejemplo: Tech Lead <-> SWE <-> QA
│   └── crisis_management.py    # Ejemplo: Support <-> SecOps <-> CEO
│
├── core/                       # ⚙️ Clases base y utilidades
│   ├── base_agent.py           # Clase abstracta para instanciar agentes
│   └── state_manager.py        # Memoria global de la empresa (Presupuesto, OKRs
│
├── docs/                       # 📚 Documentación adicional
│   ├── PATTERN_1.md            # Explicación de las reglas de Anthropic usadas
│   └── CONTRIBUTING.md         # Guía para añadir nuevos agentes
│
├── .env.example                # Variables de entorno (API Keys de OpenAI/Anthrc
├── .gitignore                  # Archivos ignorados por Git
├── LICENSE                     # Licencia (ej. MIT) para Open Source
├── README.md                   # Guía de inicio rápido
└── requirements.txt            # (O package.json) Dependencias base
```

📝 Detalle de los Archivos Clave

1. `agents/{vertical}/{nombre_agente}/system_prompt.md`

Contiene el alma del agente. Aquí se define su propósito, con quién interactúa y su `Chain of Thought` obligatorio. *Ejemplo:* No es código ejecutable, es un archivo de texto en Markdown que tu script de Python leerá y pasará como mensaje de sistema (System Message) al LLM.

2. `agents/{vertical}/{nombre_agente}/tools.xml` **(o** `.json` **)**

Contiene la definición de las herramientas bajo el esquema plano. *¿Por qué separado?* Porque permite a los desarrolladores actualizar los esquemas de las herramientas sin tocar el prompt principal de comportamiento, y facilita el *parseo* de las llamadas a funciones.

3. `workflows/`

Aquí es donde la teoría se convierte en práctica. Estos archivos (escritos en Python o TypeScript) importarían las definiciones de la carpeta `agents/` y utilizarían un framework para hacer que hablen entre ellos. *Ejemplo:* Un script que inicia una tarea, la pasa al `Tech Lead`, espera su validación, y si falla, la devuelve al `Software Engineer`.

🚀 Cómo instalar este repositorio (Guía para el README)

Si otro desarrollador quiere usar este repo, solo tendría que hacer lo siguiente:

```
# 1. Clonar el repositorio
git clone [https://github.com/tu-usuario/pom-agents-framework.git](https://github
cd pom-agents-framework

# 2. Instalar dependencias (Si usas Python/CrewAI/LangChain)
pip install -r requirements.txt

# 3. Configurar claves de API
cp .env.example .env
# Editar .env y añadir OPENAI_API_KEY o ANTHROPIC_API_KEY

# 4. Ejecutar un flujo de trabajo de prueba
python workflows/discovery_loop.py
```

🧠 ¿Por qué esta estructura?

1. **Modularidad:** Si alguien solo quiere usar el departamento de Ingeniería (`04_engineering`), puede extraer esa carpeta e ignorar el resto.
2. **Agnóstico al lenguaje:** Al guardar los prompts en `.md` y las herramientas en `.xml` o `.json`, cualquier desarrollador puede consumir esta arquitectura usando Python, Node.js, Go o Rust.

3. **Escalabilidad:** Si en el futuro creas un agente "CFO" o "Legal Analyst", tienen un lugar